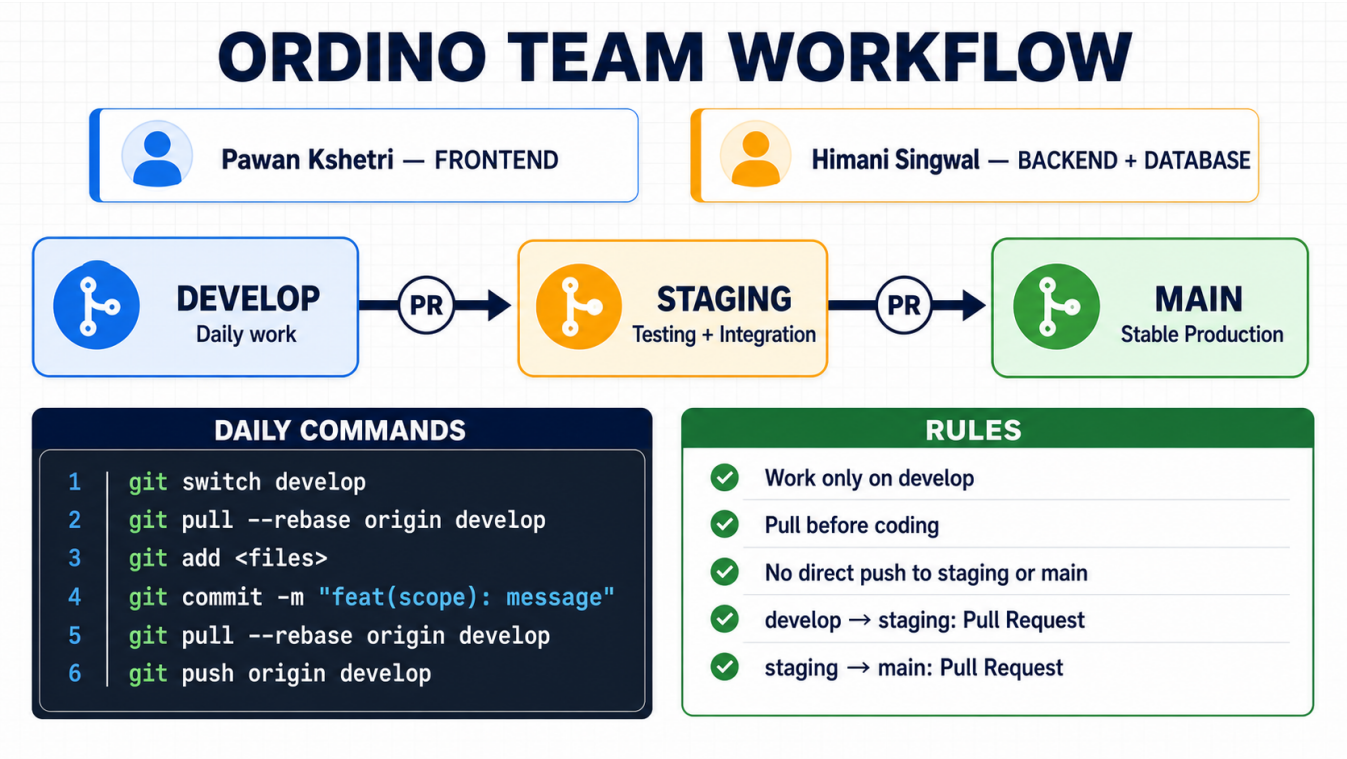
ORDINO

Git & GitHub Workflow Guide

Only the commands required by the two-developer Ordino workflow

PAWAN KSHETRI	HIMANI SINGWAL
Frontend: web, Android, and iOS	Backend APIs and database



THE WORKFLOW IN ONE SENTENCE

Daily code goes to **develop**. Tested code moves to **staging** through a pull request. Approved stable code moves to **main** through another pull request.

Repository: https://github.com/Pawankshetri11/Ordino

Prepared for the Ordino team | 25 August 2026

1. The only branch model you need

BRANCH	PURPOSE
`develop`	The daily working branch for both Pawan and Himani.
`staging`	The combined testing branch. No direct coding or normal pushes.
`main`	Stable production-ready code. No direct coding or normal pushes.

IMPORTANT

Keep exactly three branches. Do not create feature or personal branches. Normal coding happens only on **develop**; **staging** and **main** are protected promotion branches.

One-time setup: how a branch is created

```
git switch develop
git pull --ff-only origin develop
git switch -c staging
git push -u origin staging
git switch develop
```

COMMAND	WHAT IT DOES
`git switch -c staging`	Creates the local staging branch and switches to it.
`git push -u origin staging`	Publishes staging on GitHub and sets its upstream tracking branch.
`git switch develop`	Returns to develop for normal daily work.

DO NOT REPEAT THIS SETUP

The staging branch already exists on GitHub. These creation commands are included only to explain how branch creation works.

2. Real daily scenario: Pawan pushes, Himani pulls and pushes

Step A - Pawan starts frontend work

```
git switch develop
git pull --rebase origin develop
git status
# Pawan edits frontend files
git add frontend
git commit -m "feat(frontend): add login screen"
git pull --rebase origin develop
git push origin develop
```

Result: Pawan's frontend commit is now on the GitHub **develop** branch.

Step B - Himani gets Pawan's work before backend work

```
git switch develop
git pull --rebase origin develop
# Himani now has Pawan's frontend commit
# Himani edits backend and database files
git add backend
git commit -m "feat(backend): add login endpoint"
git pull --rebase origin develop
git push origin develop
```

Result: GitHub **develop** now contains Pawan's frontend work and Himani's backend work.

Step C - Pawan gets Himani's latest backend work

```
git switch develop
git pull --rebase origin develop
```

THE HABIT THAT PREVENTS MOST PROBLEMS

Pull before you start coding. After committing, pull again before you push.

3. Required Git commands and what they do

COMMAND	WHAT IT DOES
`git status`	Shows the current branch and changed or untracked files.
`git branch -vv`	Shows local branches and their remote tracking branches.
`git switch develop`	Moves your working directory to the develop branch.
`git pull --rebase origin develop`	Downloads the latest develop commits and places your local commits after them in a clean order.
`git diff`	Shows unstaged changes for review.
`git add <specific-files>`	Stages only the selected files for the next commit.
`git diff --staged`	Shows the exact staged changes before committing.
`git commit -m "message"`	Creates a local history checkpoint for the staged changes.
`git push origin develop`	Uploads local develop commits to GitHub develop.
`git log --oneline -10`	Shows the ten most recent commits in a compact format.
`git fetch origin`	Downloads remote branch information without changing working files.
`git rebase --continue`	Continues a paused rebase after conflicts are resolved and staged.
`git rebase --abort`	Safely cancels a confusing rebase and returns to the previous state.

Required GitHub pull request commands

COMMAND	WHAT IT DOES
`gh auth login`	Signs GitHub CLI into an account. Usually a one-time setup.
`gh pr create --base staging --head develop`	Creates a pull request that promotes develop into staging.
`gh pr create --base main --head staging`	Creates a pull request that promotes staging into main.

GIT VS GITHUB

A commit creates local Git history. Push uploads that history to GitHub. Pull downloads the latest GitHub commits. A pull request is GitHub's review process for merging one branch into another.

4. How to promote develop to staging

Example: the authentication milestone is ready for integrated testing.

- Pawan has pushed the frontend authentication work to develop.
- Himani has pushed the backend and database authentication work to develop.
- Both developers pulled the latest develop and checked the combined behavior.
- The complete develop snapshot is ready for staging testing.

Create the promotion pull request

```
git switch develop
git pull --rebase origin develop
git status
gh pr create --base staging --head develop \
--title "release: promote develop to staging"
```

PULL REQUEST DIRECTION

The base branch is **staging**. The compare or head branch is **develop**. This means develop is being merged into staging.

Review and merge

- The other developer reviews the combined frontend and backend changes.
- All required checks must pass.
- Use Create a merge commit on GitHub.
- Test on staging. Do not edit or push code directly to staging.

If staging testing finds a bug

```
git switch develop
git pull --rebase origin develop
# Fix the bug on develop
git add <specific-files>
git commit -m "fix(scope): fix staging issue"
git pull --rebase origin develop
git push origin develop
# Then create the develop -> staging PR again
```

5. How to promote staging to main

Main is updated only after staging testing is complete and both developers approve the release.

Create the production pull request

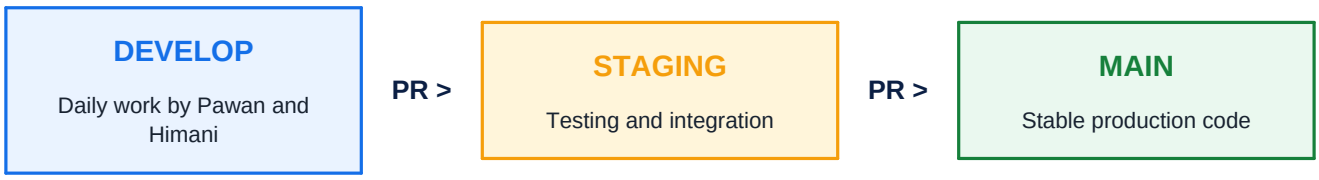
```
gh pr create --base main --head staging \
--title "release: promote staging to main"
```

PULL REQUEST DIRECTION

The base branch is **main**. The compare or head branch is **staging**. This means tested staging code is being merged into production main.

Production release checklist

- Frontend type checks and supported builds passed.
- Backend build and relevant tests passed.
- Database migrations were reviewed and tested, when the database is introduced.
- The frontend and backend API contract matches.
- No .env file, password, token, or private credential was committed.
- Both developers reviewed and approved the pull request.



NEVER SKIP STAGING

Do not merge develop directly into main. The production path is always: **develop -> staging -> main**.

6. What to do when a conflict occurs

A conflict usually means Pawan and Himani changed the same lines or the same shared file.

```
git status
# Open every conflicted file and remove conflict markers
git add <resolved-files>
git rebase --continue
git status
git push origin develop
```

NOT SURE? STOP SAFELY

If the correct resolution is unclear, run `git rebase --abort` and discuss the file with the other developer.

Commands and actions to avoid

COMMAND	WHAT IT DOES
`git push --force`	Can overwrite shared history. Do not use it in the Ordino workflow.
`Direct push to staging`	Bypasses the testing gate. Staging only receives pull requests from develop.
`Direct push to main`	Bypasses production safety. Main only receives pull requests from staging.
`git reset --hard`	Can permanently discard uncommitted work. Avoid it in this team workflow.
`git add . without review`	Can stage secrets or unrelated files. Add specific files instead.
`Commit .env or credentials`	Creates a security incident. Secrets never belong in Git.

If a push is rejected

```
git pull --rebase origin develop
# Resolve conflicts if Git asks
git push origin develop
```

7. One-page daily cheat sheet

Start of the day or task

```
git switch develop
git pull --rebase origin develop
git status
```

After coding

```
git status
git diff
git add <specific-files>
git diff --staged
git commit -m "feat(scope): clear message"
git pull --rebase origin develop
git push origin develop
```

Promote a milestone to staging

```
gh pr create --base staging --head develop \
--title "release: promote develop to staging"
```

Promote approved staging to main

```
gh pr create --base main --head staging \
--title "release: promote staging to main"
```

FINAL MEMORY RULE

Pawan pushes to develop -> Himani pulls develop -> Himani pushes to develop -> pull request from develop to staging -> testing -> pull request from staging to main.

Keep this PDF beside the repository until the workflow becomes a habit.